

30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 22)

**“Tick Rate, Tick Interrupt,
and Tickless Idle”**



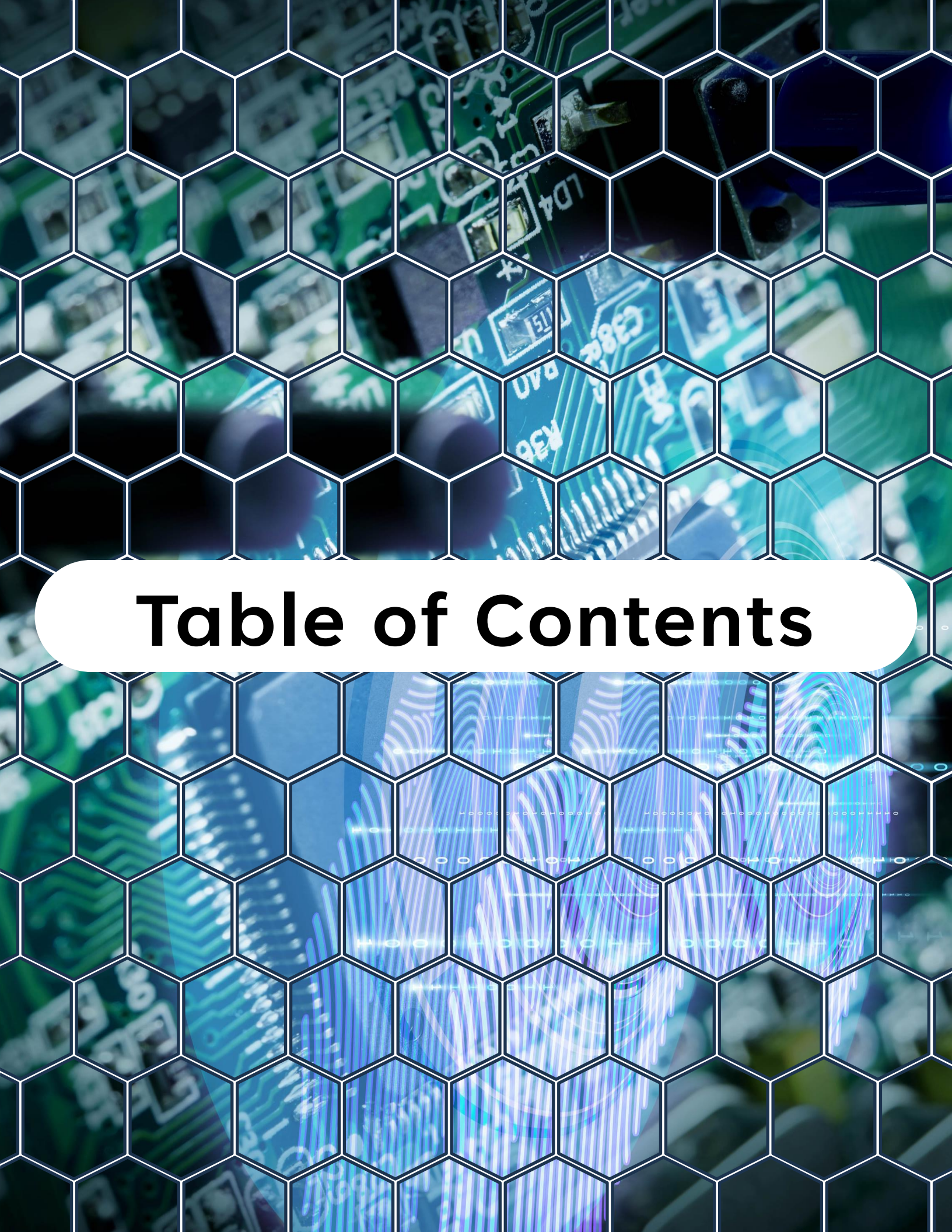
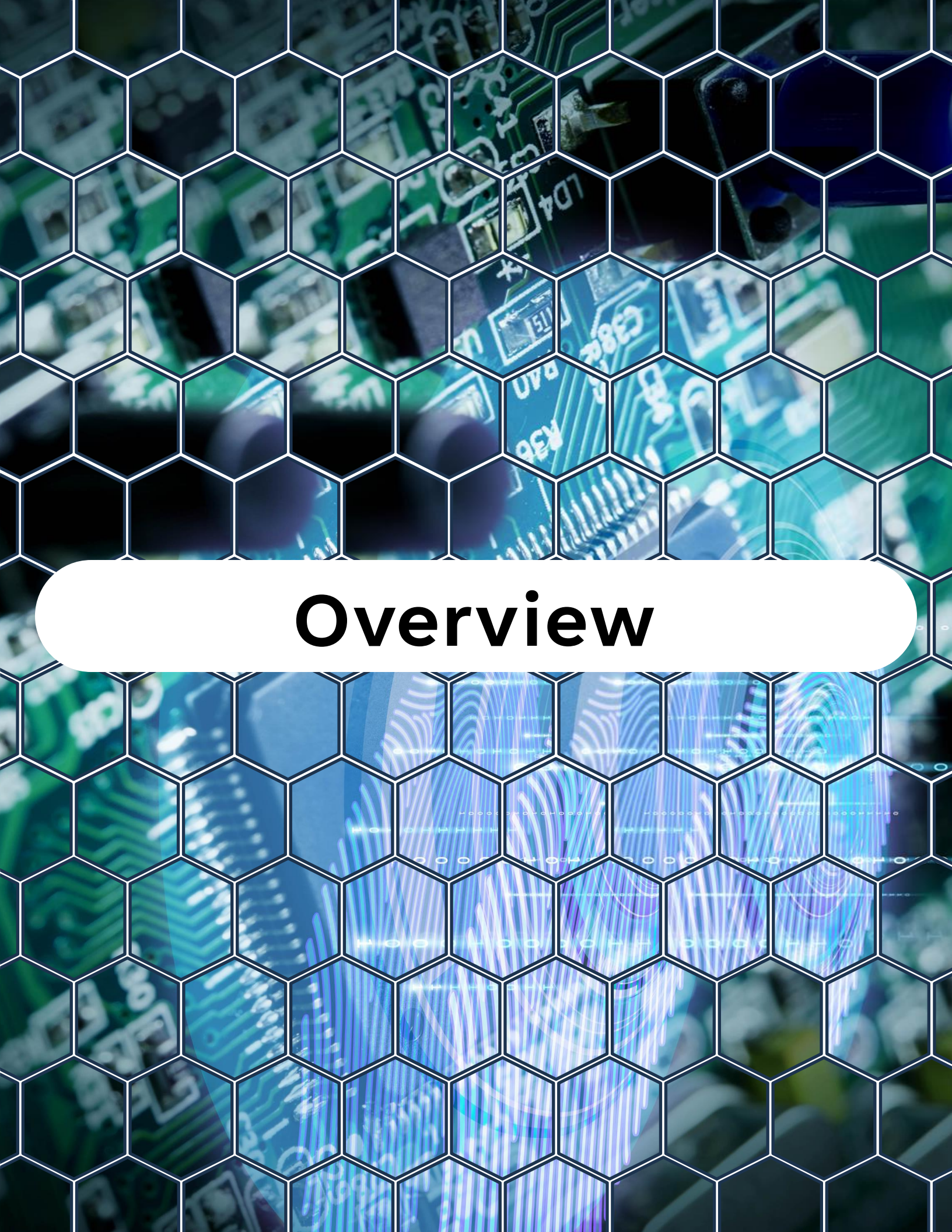


Table of Contents

Table of Contents

1. Overview
2. What Is the System Tick?
3. Tick Rate (configTICK_RATE_HZ)
4. Tick Interrupt
5. Tickless Idle
6. Example – Observing Tick Hook
7. Best Practices
8. Summary
9. Challenge for Today



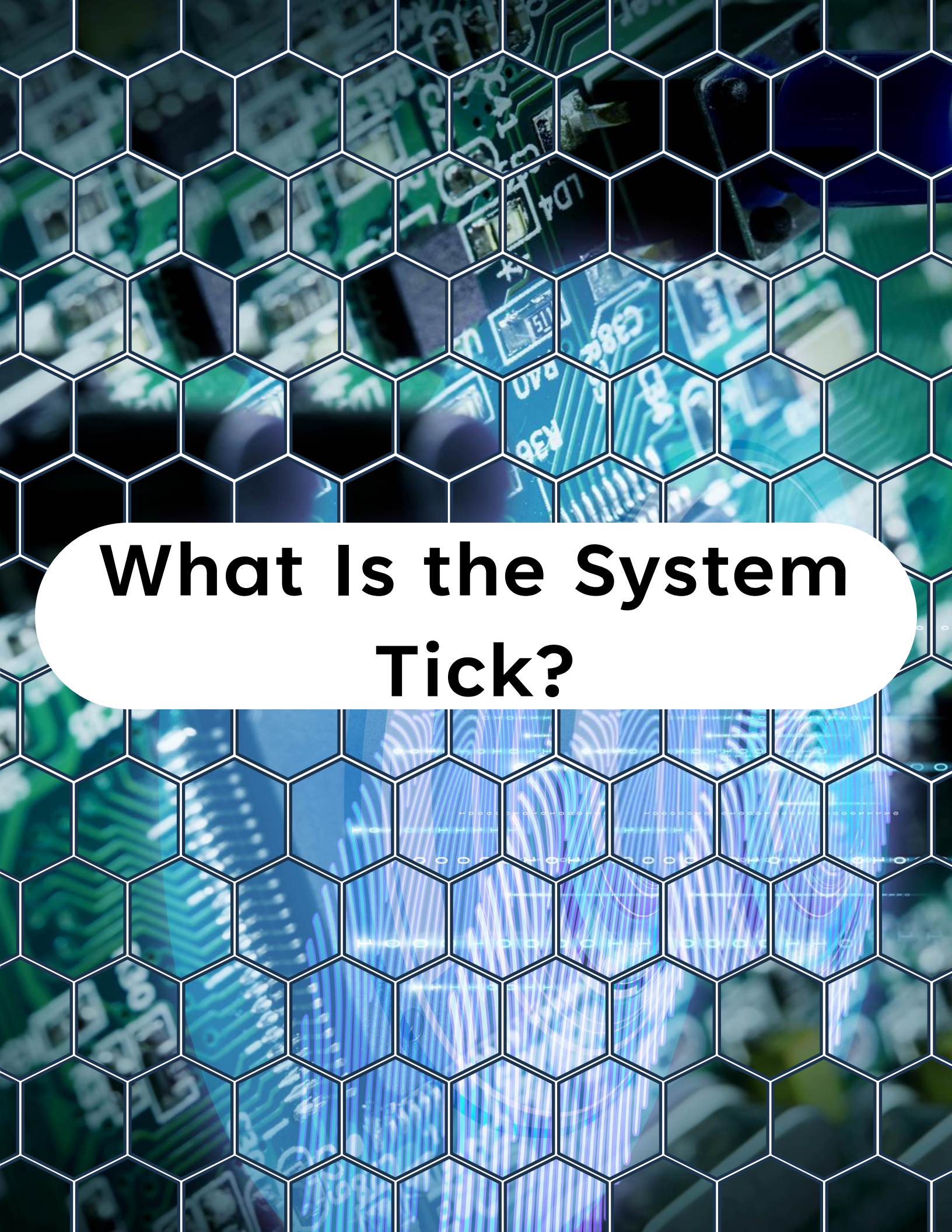
Overview

Overview

On **Day 22**, you'll learn:

- How FreeRTOS uses the **system tick** for task scheduling
- How to configure the **tick rate (frequency)** in ESP-IDF
- The role of the **tick interrupt** in preemptive multitasking
- How **tickless idle mode** works and how it saves power
- A practical ESP32 example to observe tick timing and tickless idle
- Best practices when tuning tick rate and idle mode

By the end of this lesson, you'll understand how the **heartbeat of FreeRTOS (the tick)** drives task scheduling and how to optimize it for **performance or power efficiency**.



What Is the System Tick?

What Is the System Tick?

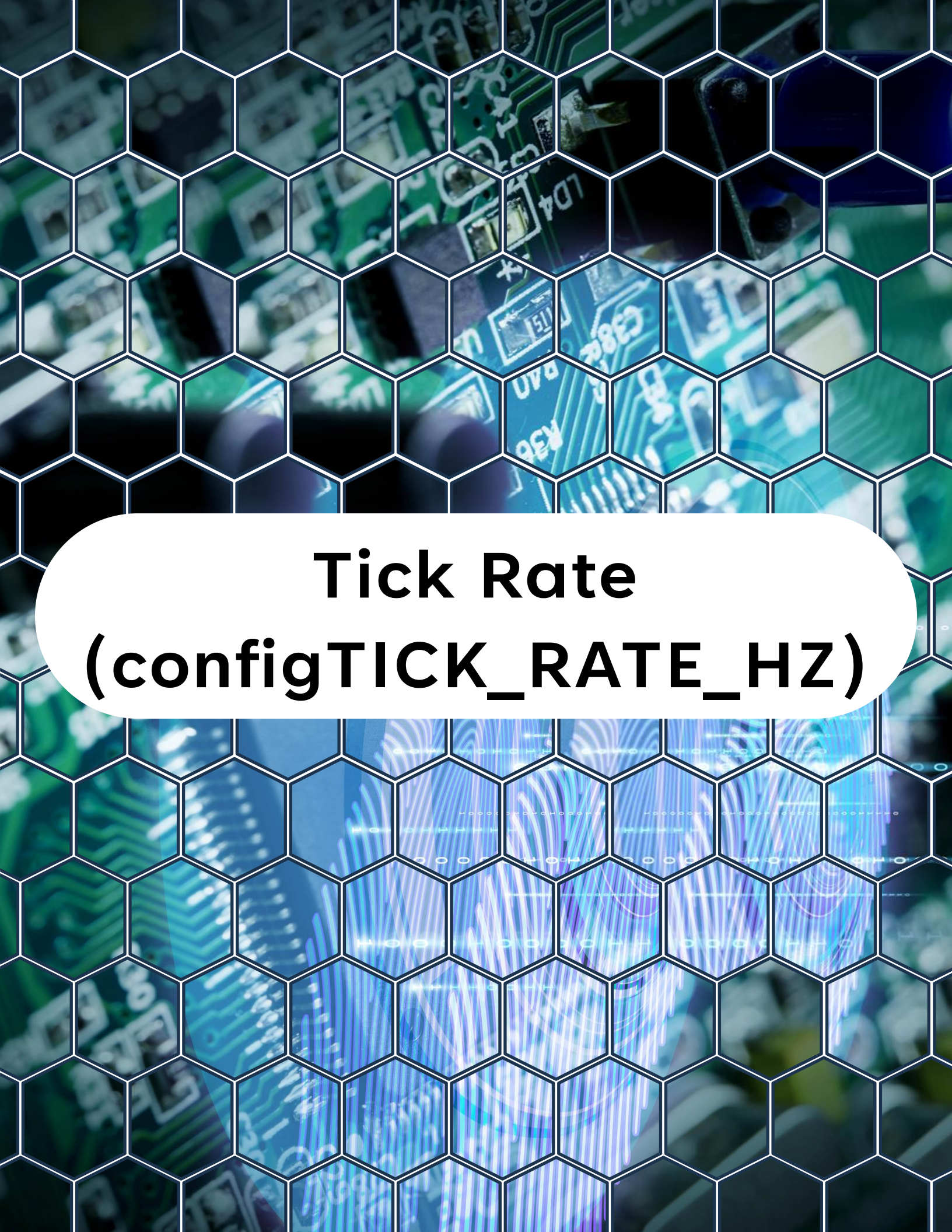
The **system tick** is a periodic interrupt that

FreeRTOS uses to:

- Update system time.
- Manage delays (`vTaskDelay`, `vTaskDelayUntil`).
- Decide when to switch between tasks (scheduler).



Think of it as the **metronome** of FreeRTOS.



Tick Rate **(configTICK_RATE_HZ)**

Tick Rate

(`configTICK_RATE_HZ`)

- The tick rate defines how many ticks occur per second.

- Configurable in ESP-IDF:

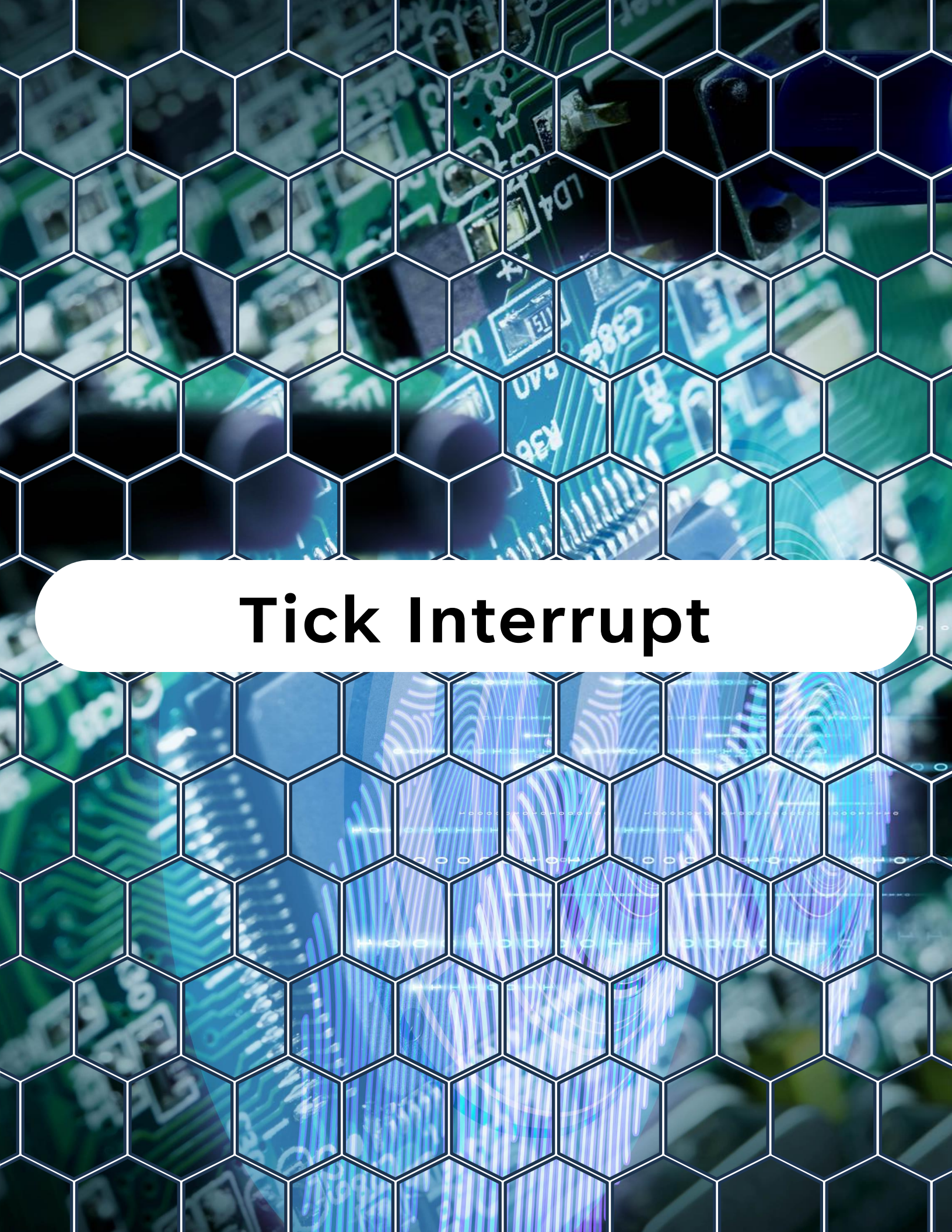
idf.py menuconfig → Component config →

FreeRTOS → Tick rate (Hz)

- Common values:
 - **100 Hz (10 ms resolution)** – default in ESP-IDF.
 - **1000 Hz (1 ms resolution)** – finer timing, but more CPU overhead.



Trade-off: Higher tick rate = more precise timing, but higher CPU usage.



Tick Interrupt

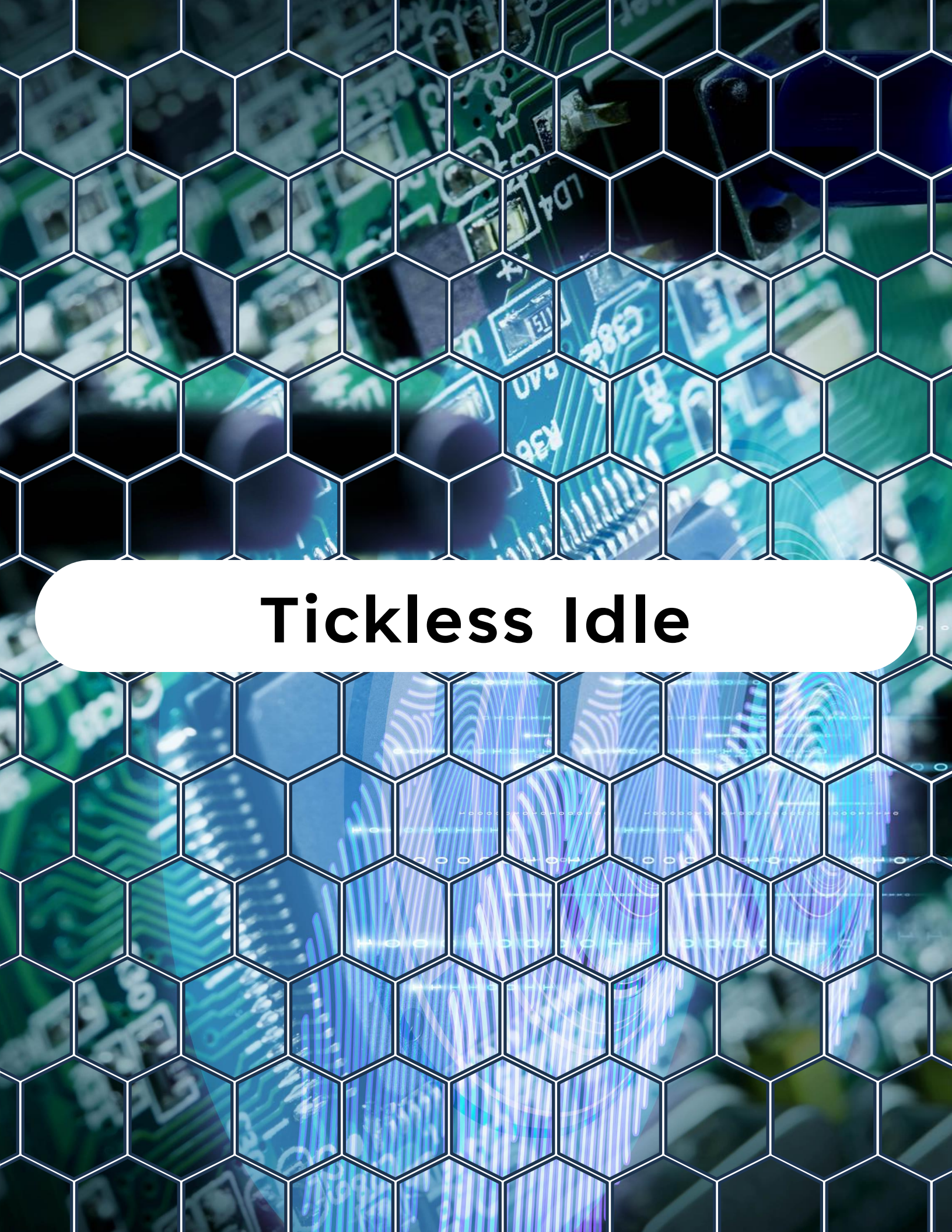
Tick Interrupt

The ESP32 hardware generates the tick interrupt using a **timer peripheral**.

Tick Interrupt Tasks:

- Increments the **tick count**.
- Checks if any delayed tasks should unblock.
- Decides if a context switch is needed.

 The tick interrupt is central to **preemptive scheduling**: it ensures higher-priority ready tasks preempt lower-priority ones at the next tick.



Tickless Idle

Tickless Idle

Tickless Idle is a power-saving feature:

- Normally, ticks occur at a fixed rate (e.g., every 10 ms).
- In idle mode, ticks may be **suppressed** if no tasks are scheduled to wake soon.
- The hardware timer is reprogrammed to fire only when the next task needs to wake.

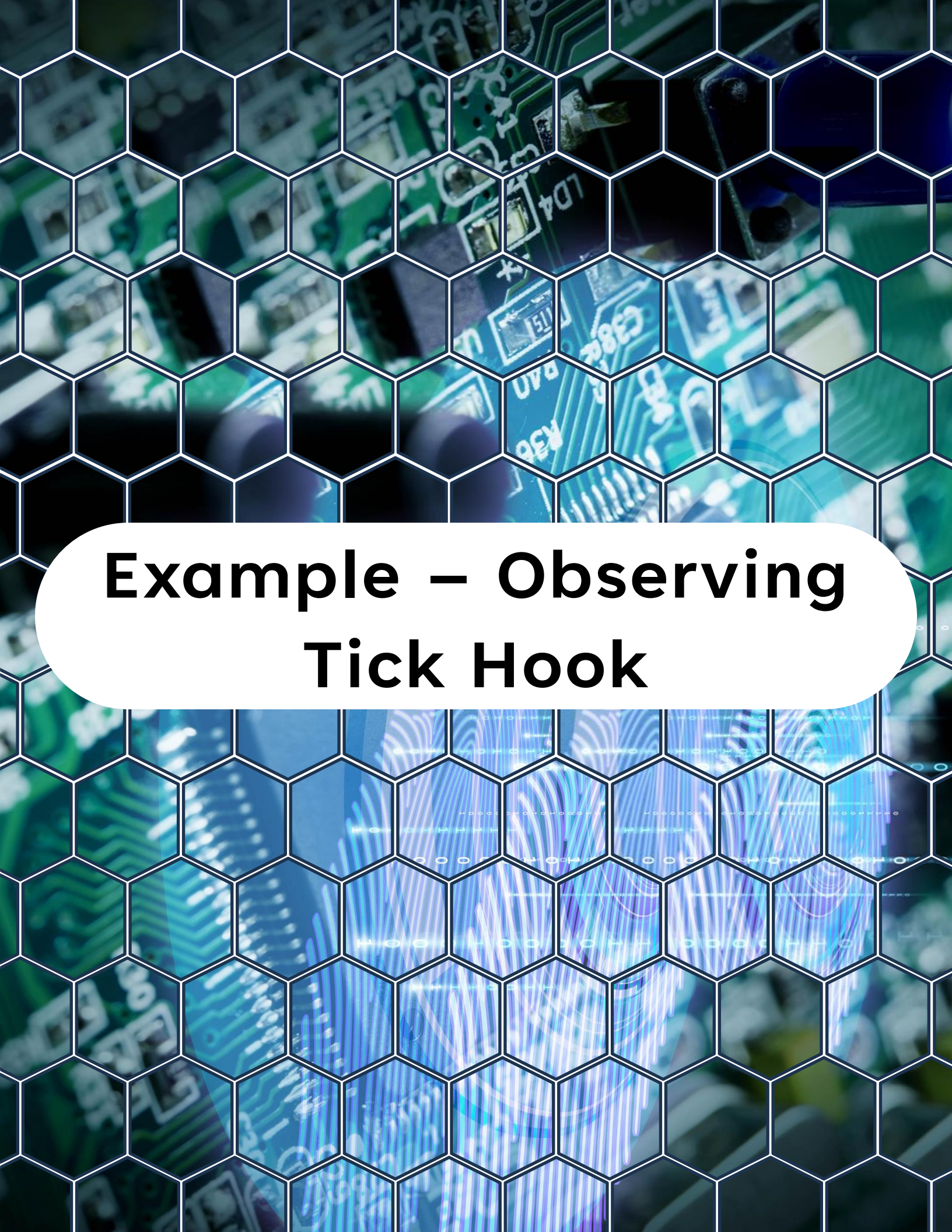


Benefits:

- Reduces CPU wakeups.
- Saves power → especially important for battery-powered ESP32 IoT devices.

Enable in menuconfig:

Component config → FreeRTOS → Tickless
idle support



Example – Observing Tick Hook

Example – Observing Tick Hook

Code Example

```
1  #include <stdio.h>
2  #include <inttypes.h>
3  #include "freertos/FreeRTOS.h"
4  #include "freertos/task.h"
5  #include "driver/gpio.h"
6  #include "esp_log.h"
7
8  #define TAG "TICK_HOOK_DEMO"
9  #define LED_GPIO GPIO_NUM_2 // adjust for your board
10
11 // Task handle that will be notified from the tick hook
12 static TaskHandle_t s_tick_task = NULL;
13
14 // Optional: track approx. ms since boot based on tick period
15 static volatile uint32_t s_ms_since_boot = 0;
16
17 /*
18  * Tick hook: called by FreeRTOS on EVERY tick (from the tick ISR).
19  * Keep it tiny: no blocking, no logging, only ISR-safe calls.
20  */
21 void IRAM_ATTR vApplicationTickHook(void)
22 {
23     // Accumulate milliseconds using the known tick period
24     s_ms_since_boot += portTICK_PERIOD_MS;
25
26     // Notify once every 500 ms
27     static uint32_t acc_ms = 0;
28     acc_ms += portTICK_PERIOD_MS;
29
30     if (acc_ms >= 500) {
31         acc_ms -= 500;
32
33         BaseType_t higher_woken = pdFALSE;
34         if (s_tick_task) {
35             // Wake the task (one count per half-second)
36             vTaskNotifyGiveFromISR(s_tick_task, &higher_woken);
37         }
38         if (higher_woken) {
```

Example – Observing Tick Hook

```
17 /*
18  * Tick hook: called by FreeRTOS on EVERY tick (from the tick ISR).
19  * Keep it tiny: no blocking, no logging, only ISR-safe calls.
20  */
21 void IRAM_ATTR vApplicationTickHook(void)
22 {
23     // Accumulate milliseconds using the known tick period
24     s_ms_since_boot += portTICK_PERIOD_MS;
25
26     // Notify once every 500 ms
27     static uint32_t acc_ms = 0;
28     acc_ms += portTICK_PERIOD_MS;
29
30     if (acc_ms >= 500) {
31         acc_ms -= 500;
32
33         BaseType_t higher_woken = pdFALSE;
34         if (s_tick_task) {
35             // Wake the task (one count per half-second)
36             vTaskNotifyGiveFromISR(s_tick_task, &higher_woken);
37         }
38         if (higher_woken) {
39             portYIELD_FROM_ISR();
40         }
41     }
42 }
43
44 // Task that waits for notifications from the tick hook
45 static void tick_consumer_task(void *arg)
46 {
47     bool led_on = false;
48
49     for (;;) {
50         // Block until the tick hook gives us a notification (every 500 ms)
51         ulTaskNotifyTake(/*clearOnExit=*/pdTRUE, /*wait=*/portMAX_DELAY);
52
53         // Safe to do work here (task context)
54         ESP_LOGI(TAG, "Half-second tick: ~%" PRIu32 " ms since boot",
55                 s_ms_since_boot);
56
57         led_on = !led_on;
58         gpio_set_level(LED_GPIO, led_on);
59     }
```


Example – Observing Tick Hook

```
44 // Task that waits for notifications from the tick hook
45 static void tick_consumer_task(void *arg)
46 {
47     bool led_on = false;
48
49     for (;;) {
50         // Block until the tick hook gives us a notification (every 500 ms)
51         ulTaskNotifyTake(/*clearOnExit=*/pdTRUE, /*wait=*/portMAX_DELAY);
52
53         // Safe to do work here (task context)
54         ESP_LOGI(TAG, "Half-second tick: ~%" PRIu32 " ms since boot",
55                 s_ms_since_boot);
56
57         led_on = !led_on;
58         gpio_set_level(LED_GPIO, led_on);
59     }
60 }
61
62 // Main application entry point
63 void app_main(void)
64 {
65     // Configure LED
66     gpio_config_t io = {
67         .pin_bit_mask = (1ULL << LED_GPIO),
68         .mode = GPIO_MODE_OUTPUT,
69         .pull_up_en = GPIO_PULLUP_DISABLE,
70         .pull_down_en = GPIO_PULLDOWN_DISABLE,
71         .intr_type = GPIO_INTR_DISABLE
72     };
73     gpio_config(&io);
74     gpio_set_level(LED_GPIO, 0);
75
76     // Create the task to be notified by the tick hook
77     xTaskCreate(tick_consumer_task, "TickConsumer", 2048, NULL, 6, &s_tick_task);
78
79     ESP_LOGI(TAG, "Tick hook demo started. TICK_RATE_HZ=%d, tick=%d ms",
80             configTICK_RATE_HZ, (int)portTICK_PERIOD_MS);
81 }
```

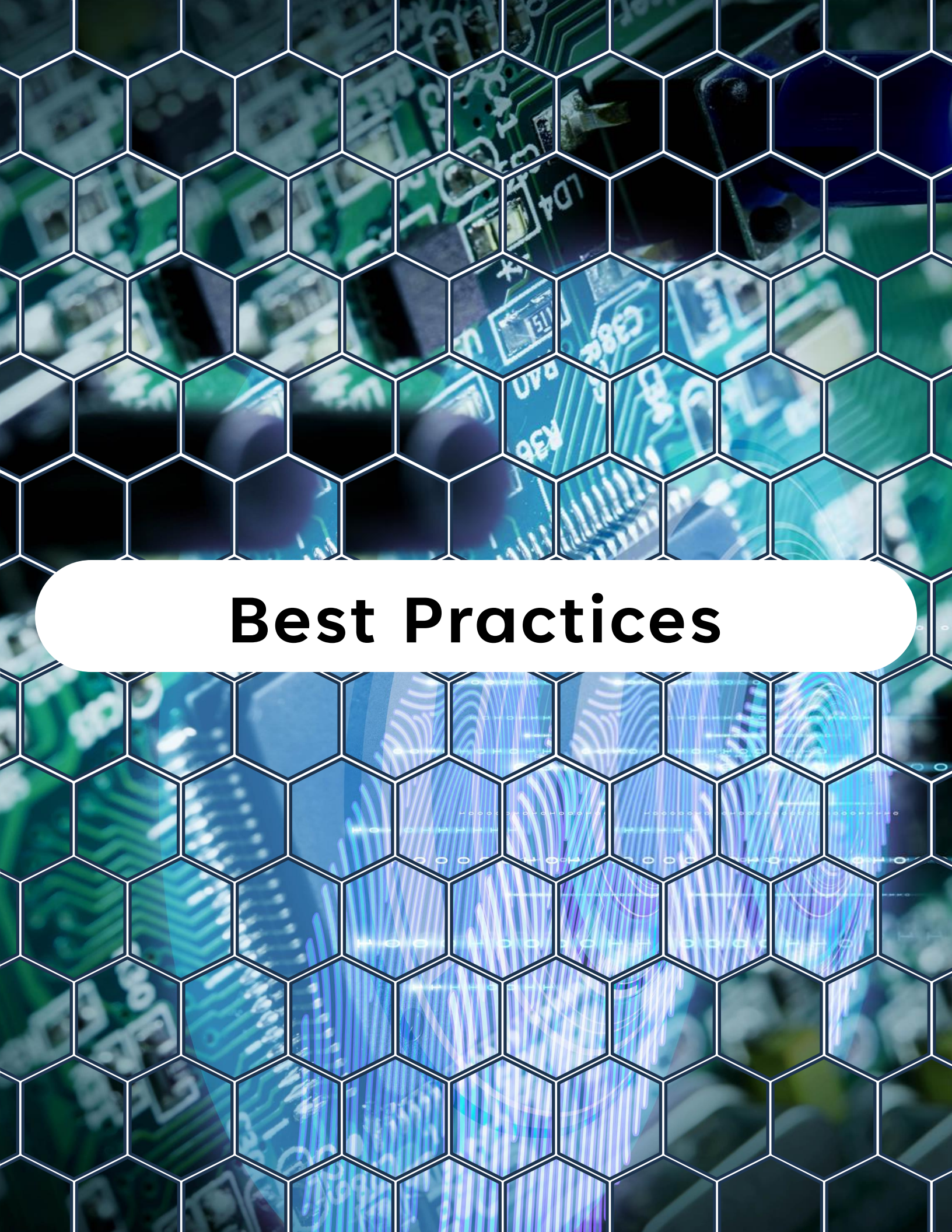
Example – Observing Tick Hook

Expected Behavior

- Every 500 ms, the LED toggles and you'll see a log like:

```
I (1500) TICK_HOOK_DEMO: Half-second tick: ~1500  
ms since boot
```

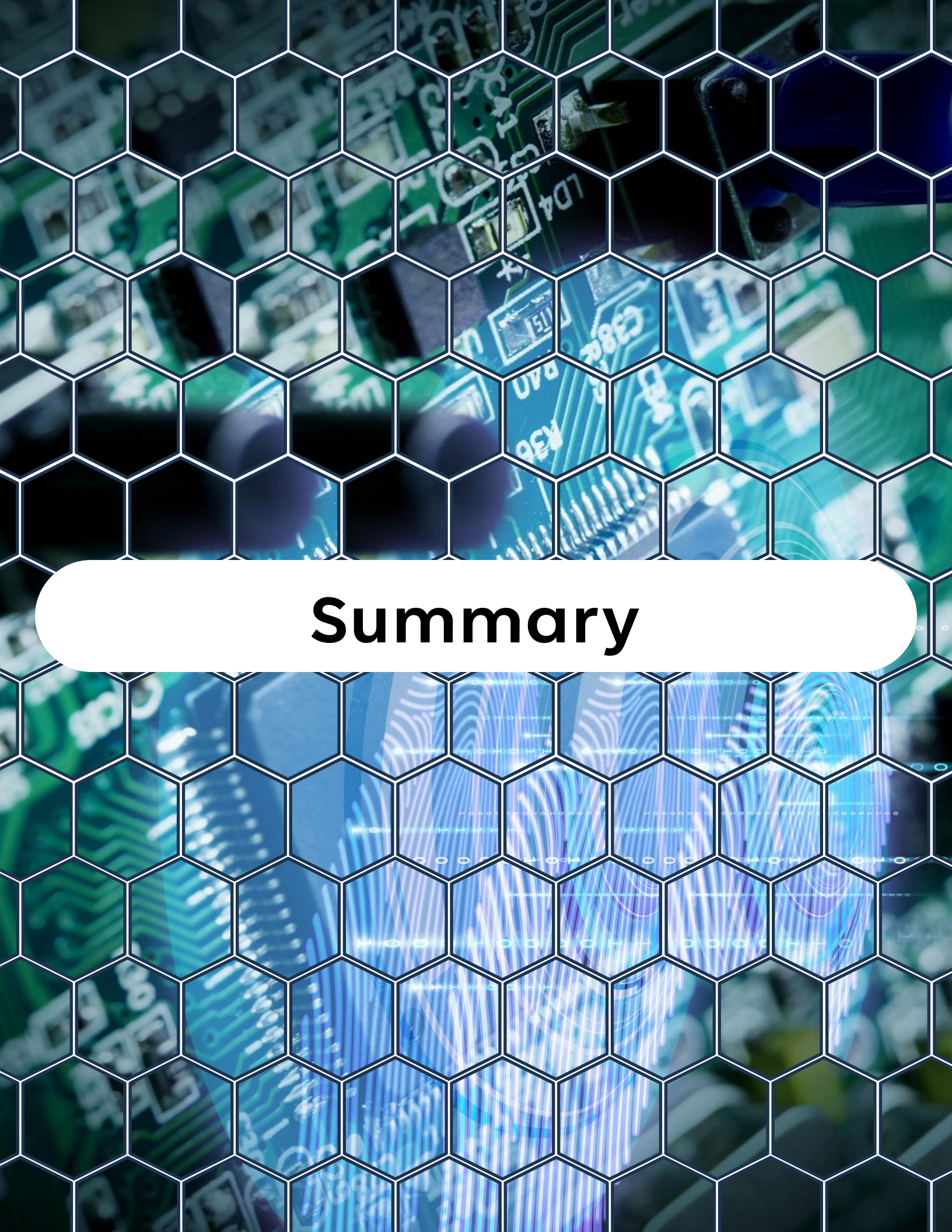
- Timing is derived from the RTOS tick (portTICK_PERIOD_MS), no extra interrupt needed beyond the system tick.



Best Practices

Best Practices

- ✓ Choose tick rate based on application: 100
 - Hz → sufficient for many IoT apps.
 - 1000 Hz → required for fine-grained real-time tasks.
- ✓ Enable **tickless idle** for battery-powered or low-power applications.
- ✓ Avoid setting tick rate too high → increases CPU load and power consumption.
- ✓ Use **hardware timers** or **esp_timer** for sub-millisecond precision (don't rely solely on FreeRTOS ticks).
- ✓ Profile your app: measure CPU idle percentage to know if tickless idle helps.



Summary

Summary

- The **system tick** is the heartbeat of FreeRTOS scheduling.
- **Tick rate** sets timing resolution vs CPU overhead trade-offs.
- **Tick interrupt** drives delays, task wakeups, and preemption.
- **Tickless idle** saves power by suppressing unnecessary ticks.
- ESP32 apps can balance **precision** and **efficiency** with proper tick settings.



Challenge for Today

Challenge for Today



Modify today's project by:

1. Change the notification rate:

- Adjust the accumulator in `vApplicationTickHook()` so the task is notified every 100 ms instead of 500 ms.
- Observe how this affects LED toggling and logging.

2. Add a second task:

- Create a second task that is also notified by the tick hook, but only every 1000 ms (1 second).
- Use a separate counter in the hook to notify each task at different intervals.



"Thanks for watching!

*If you enjoyed this video, make sure to hit
the like button and subscribe to stay
updated with my latest content.*

*Don't forget to check out my other videos
for more tips and tutorials on Embedded
C, Python, hardware designs, etc. Keep
exploring, keep learning, and I'll see you in
the next video!"*

<https://www.linkedin.com/in/yamil-garcia>
<https://www.youtube.com/@LearningByTutorials>
<https://github.com/god233012yamil>